🎯 Interview Questions & Answers

AI Document Processing & RAG System

==========

📝 PROJECT DESCRIPTION (7-8 lines — memorize this)

"I built a full-stack AI Document Processing and RAG system using FastAPI for the backend and Next.js for the frontend, with PostgreSQL 17 and pgvector as the vector database. The system allows users to upload PDF documents which are automatically processed through a pipeline — text is extracted using PyMuPDF, scanned documents go through Tesseract OCR, then the text is split into chunks using a recursive text splitter. Each chunk is converted into a 384-dimensional vector using sentence-transformers running locally, and stored in PostgreSQL using the pgvector extension with an HNSW index for fast similarity search. The AI features are powered by Groq LLM (qwen model) integrated through LangChain LCEL chains, and a LangGraph CRAG agent that retrieves document chunks, grades their relevance, rewrites queries if needed, and generates answers with page citations. Users can ask questions about documents, get AI summaries, extract structured fields like company name or revenue, and download PDF reports. The entire system uses JWT authentication with role-based permissions and supports multi-tenant architecture through a Company model."

🔷 SECTION 1 — PROJECT OVERVIEW

Q1. What is this project and what problem does it solve? It is an AI-powered document processing system that lets users upload PDFs and ask questions about them using RAG. It solves the problem of manually reading large documents by providing instant AI-generated answers with page citations.

Q2. What is RAG and why did you use it? RAG stands for Retrieval-Augmented Generation. Instead of relying on the LLM's training data, we retrieve relevant document chunks first and feed them as context. This prevents hallucinations and ensures answers are grounded in the actual document.

Q3. What are the main features of this project? PDF upload with OCR, semantic vector search, RAG Q&A with citations, LangGraph CRAG agent, hierarchical AI summarization, structured field extraction, and downloadable PDF reports — all behind JWT authentication.

Q4. What is the tech stack? Backend: FastAPI, PostgreSQL 17, pgvector, LangChain, LangGraph, Groq LLM, sentence-transformers. Frontend: Next.js 15. Infrastructure: Alembic for migrations, ReportLab for PDF generation.

 SECTION 2 — FASTAPI & BACKEND

Q5. Why did you choose FastAPI over Django? FastAPI is async-first, which means it handles multiple requests without blocking. For AI operations like embedding generation and LLM calls that take time, async prevents the server from freezing on each request.

Q6. How is your project structured in FastAPI? It follows a clean layered architecture — Routes (api/v1) → Services (business logic) → Repositories (DB queries) → Models (SQLAlchemy ORM). No business logic lives in routes.

Q7. What is the Repository pattern and why did you use it? Repository pattern separates DB queries from business logic. All raw SQL and SQLAlchemy queries live in repository files, so services stay clean and testable without touching the database directly.

Q8. How did you handle async in FastAPI? All route handlers and service methods are `async def`. Synchronous operations like embedding generation and Groq API calls are wrapped in `asyncio.to_thread()` so they run in a thread pool without blocking the event loop.

Q9. What is the purpose of `asyncio.to_thread()`? sentence-transformers and Groq SDK are synchronous libraries. Calling them directly in async routes

blocks the event loop. `asyncio.to_thread()` runs them in a separate thread, keeping the API responsive.

Q10. How does background document processing work? When a PDF is uploaded, the route returns `202 Accepted` immediately and adds the processing task to FastAPI's `BackgroundTasks`. The pipeline (extract → OCR → chunk → embed → store) runs in the background while the user polls the status endpoint.

Q11. What is Pydantic v2 and how did you use it? Pydantic v2 validates all request/response data using Python type hints. We use it in schemas for login, register, document upload, and extraction requests — it auto-validates types, raises clear errors, and serializes responses.

Q12. How does your exception handling work? We have a global `register_exception_handlers()` function that catches all errors and returns a consistent JSON format: `{"success": false, "error": {"code": "...", "message": "..."}}`. Custom exceptions like `NotFoundError` or `LLMError` map to specific HTTP codes.

🔷 SECTION 3 — DATABASE & pgvector

Q13. What is pgvector and why did you use it instead of a separate vector DB? pgvector is a PostgreSQL extension that adds a `VECTOR` column type with cosine similarity search. Using it keeps all data — metadata and vectors — in one database, simplifying the architecture without needing Pinecone or Chroma.

Q14. How do you store embeddings in PostgreSQL? The `document_chunks` table has a `embedding VECTOR(384)` column. Each text chunk gets a 384-dimensional float vector from sentence-transformers stored in this column alongside the chunk content and page number.

Q15. What is an HNSW index and why did you create it? HNSW (Hierarchical Navigable Small World) is a graph-based approximate nearest neighbor index. It makes vector similarity searches extremely fast — milliseconds instead of seconds — even with millions of vectors.

Q16. How does cosine similarity search work in your project? We use the pgvector <=> operator which computes cosine distance. The query `1 -`

`(embedding <=> query_vector)` gives similarity from 0 to 1. We filter results below a relevance threshold and return the top-K most similar chunks.

Q17. What is Alembic and how did you use it? Alembic is the migration tool for SQLAlchemy. When we change a model — add a column, create a table — we run `alembic revision --autogenerate` which detects changes and generates a migration file, then `alembic upgrade head` applies it.

Q18. How did you handle the async SQLAlchemy setup? We use `create_async_engine` with `asyncpg` driver for the app, and a separate `psycopg2` sync connection only for Alembic migrations. The `get_db()` dependency injects an `AsyncSession` into every route.

Q19. What is the multi-tenant design in your project? Every user belongs to a `Company`. Documents are scoped to a company — users can only see their own company's documents. The `company_id` foreign key on both `User` and `Document` enforces this isolation.

🔷 SECTION 4 — LangChain & LCEL

Q20. What is LangChain and why did you use it? LangChain is a framework for building LLM applications. We used it for LCEL chains (composable pipelines), prompt templates, and as the backbone for LangGraph agents. It standardizes how we call LLMs and parse outputs.

Q21. What is LCEL (LangChain Expression Language)? LCEL is a declarative way to chain LangChain components using the pipe `|` operator. Example: `chain = prompt | llm | parser`. It makes chains composable, streamable, and easy to add retry or fallback logic.

Q22. What LangChain components did you use? `ChatPromptTemplate` for structured prompts, `ChatGroq` for Groq LLM integration, `StrOutputParser` for text output, `JsonOutputParser` for structured JSON output, and `RecursiveCharacterTextSplitter` for document chunking.

Q23. What is `ChatGroq` in LangChain? `ChatGroq` is LangChain's wrapper around the Groq API. It implements the `BaseChatModel` interface so it works

seamlessly with LCEL chains and LangGraph agents without changing any other code.

Q24. What is `JsonOutputParser` and when did you use it?
`JsonOutputParser` parses LLM output as JSON automatically. We use it for extraction chains where the LLM returns structured data like `{"company_name": {"value": "Acme", "status": "found"}}`.

Q25. What is LangSmith and did you use it? LangSmith is LangChain's observability platform. When `LANGCHAIN_TRACING_V2=true` is set, every LLM call, chain step, and agent node is automatically logged to Smith for debugging and performance monitoring.

Q26. What is `HuggingFaceEmbeddings` in LangChain? It is LangChain's wrapper around sentence-transformers models. We use it to generate 384-dimensional vectors locally — no API key needed, no cost, runs entirely on the machine.

🔷 SECTION 5 — LangGraph & AGENTS

Q27. What is LangGraph? LangGraph is a framework for building stateful AI agents as directed graphs. Each node is a function (retrieve, grade, generate), edges define flow, and the `StateGraph` carries data between nodes like messages and retrieved documents.

Q28. What is the CRAG agent and how does it work? CRAG stands for Corrective RAG. It retrieves document chunks, grades each one for relevance, and if the quality is poor, rewrites the query and retrieves again before generating an answer. This gives much better answers than simple RAG.

Q29. What is `StateGraph` in LangGraph? `StateGraph` is the core LangGraph class. You define a `TypedDict` for the state, add nodes (functions), add edges (flow), and compile it into a runnable graph. State flows through every node and accumulates results.

Q30. What is `TypedDict` and why is it used in LangGraph? `TypedDict` defines the state schema with Python type hints. LangGraph uses it to know what

fields are in the state. `Annotated[list, add]` means list fields are appended across nodes instead of replaced.

Q31. What are the nodes in your RAG agent? Four nodes: `retrieve` (pgvector search), `grade_docs` (LLM scores each chunk for relevance 0-1), `rewrite_query` (LLM rewrites poor query), and `generate` (build context and get Groq answer with citations).

Q32. What are conditional edges in LangGraph? Conditional edges route to different nodes based on the state. After `grade_docs`, we check: if enough good docs → `generate`, if poor docs and not rewritten → `rewrite_query`, if max iterations reached → force `generate`.

Q33. What is the multi-tool document agent? It's a second LangGraph agent that first classifies the user's message intent (answer_question / summarize / extract_fields / search) then routes to the correct service. So users can just type naturally without picking an endpoint.

Q34. What is `ainvoke()` in LangGraph? `ainvoke()` is the async version of running a LangGraph graph. Since our nodes are `async def` functions, we call `await graph.ainvoke(initial_state)` which runs the full graph asynchronously.

Q35. What fallback did you add to the agents? Both agents have a `try/except` block. If LangGraph fails for any reason, they fall back to a simple direct RAG call — retrieve chunks and generate answer without the graph — so the API never returns a 500 error.

🔷 SECTION 6 — DOCUMENT PROCESSING

Q36. How does PDF text extraction work? PyMuPDF (fitz) opens the PDF and extracts text page by page using `page.get_text()`. If a page has fewer than 50 characters, it's flagged as needing OCR because it's likely a scanned image.

Q37. How does OCR work in your project? For scanned pages, we convert them to images using PyMuPDF at 300 DPI, preprocess with OpenCV (grayscale, denoise, adaptive threshold, deskew), then run Tesseract which returns text with confidence scores per word.

Q38. What is chunking and why is it important? Chunking splits long documents into smaller pieces that fit in the LLM context window. We use RecursiveCharacterTextSplitter with 800-word chunks and 150-word overlap so no sentence is cut mid-thought at chunk boundaries.

Q39. What is chunk overlap and why does it matter? Overlap means the last 150 words of one chunk are repeated at the start of the next. This prevents losing context at boundaries — if a key sentence spans two chunks, it appears in both ensuring it's retrieved.

Q40. What is sentence-transformers and which model did you use? sentence-transformers is a Python library for generating semantic text embeddings. We use `all-MiniLM-L6-v2` which produces 384-dimensional vectors. It runs locally on CPU, requires no API key, and is optimized for semantic similarity.

 SECTION 7 — GROQ LLM

Q41. What is Groq and why did you choose it? Groq is an AI inference platform using custom GroqChip hardware. It offers extremely fast LLM inference — much faster than OpenAI — with a generous free tier. We use the `qwen/qwen3.8-27b` model.

Q42. What LLM tasks does Groq handle in your project? Five tasks: RAG answer generation, document grading (is chunk relevant?), query rewriting (improve poor queries), hierarchical summarization (chunk batches then final), and structured field extraction with JSON output.

Q43. What is JSON mode in Groq? When `response_format={'type': 'json_object'}` is set, Groq guarantees the response is valid JSON. We use this for extraction and grading so we can safely call `json.loads()` on the output without try/except parsing errors.

Q44. How did you make Groq calls non-blocking? The Groq Python SDK is synchronous. We wrap calls in `asyncio.to_thread()` which runs the blocking call in a thread pool. This frees the FastAPI event loop to handle other requests while waiting for Groq to respond.

🔷 SECTION 8 — AUTH & SECURITY

Q45. How does JWT authentication work in your project? On login, we create two JWTs: an access token (10 hours) and a refresh token (15 days). All protected routes check the `Authorization: Bearer <token>` header via the `get_current_user` FastAPI dependency.

Q46. What is the role-based permission system? There are 17 permission codes like `can_upload_document`, `can_chat`, `can_summarize`. Each Role has many Permissions via a many-to-many table. The `user.has_perm('code_name')` method checks if the user's role has that permission.

Q47. How does the `Depends()` pattern work for auth? FastAPI's `Depends(get_current_user)` injects the authenticated user into any route. We also have `Depends(require_superuser)` and `Depends(require_permission('code_name'))` factory functions for fine-grained access control.

Q48. How are passwords stored? Passwords are hashed using bcrypt with 12 rounds via passlib. We never store plain passwords. `verify_password(plain, hashed)` compares during login, and `hash_password(plain)` creates the stored hash on register.

🔷 SECTION 9 — FILES & THEIR ROLES

Q49. What does `document_worker.py` do? It is the async background pipeline that runs after upload: extract text → OCR if needed → save pages to DB → chunk text → generate embeddings → store chunks with vectors → mark document as READY.

Q50. What does `llm_service.py` do? It is the central LLM interface. All Groq API calls go through it — answer_from_context, grade_document, rewrite_query, summarize_chunks, final_summary, extract_fields. Every method is async using `asyncio.to_thread()`.

Q51. What does `rag_service.py` do? It orchestrates the full RAG pipeline. It checks if the document is ready, then either runs the LangGraph CRAG agent (if `USE_AGENT_MODE=True`) or falls back to simple retrieve → generate.

Q52. What does `vector_search_service.py` do? It takes a text query, generates its embedding asynchronously, runs a pgvector cosine similarity SQL query, filters by relevance threshold, and returns ranked `SearchResult` objects with chunk content and page numbers.

Q53. What does `summary_service.py` do? It implements hierarchical summarization: loads all document chunks, summarizes them in batches of 5 using Groq, combines the batch summaries, then calls Groq once more to generate a structured final summary with key_points, risks, and conclusion.

Q54. What does `extraction_service.py` do? It takes a list of field names, finds relevant chunks via vector search, builds context, sends it to Groq with instructions to extract each field, and returns structured results with `value` and `status` (found/not_found/uncertain) per field.

Q55. What does `langchain_setup.py` do? It initializes all LangChain singletons using `@lru_cache` — ChatGroq instances (main, summary, json), HuggingFaceEmbeddings, and LangSmith tracing setup. All services import from here instead of creating new clients.

Q56. What does `report_service.py` do? It generates downloadable PDF reports using ReportLab. It creates a styled PDF with document info on a cover page, executive summary section, key points, important numbers, risks, and extracted data in a formatted table.

 SECTION 10 — KEY TERMINOLOGY

Q57. What is a vector embedding? A vector embedding is a list of numbers (e.g., 384 floats) that represents the semantic meaning of text. Similar sentences have vectors that are mathematically close to each other, enabling semantic search.

Q58. What is cosine similarity? Cosine similarity measures the angle between two vectors. A score of 1.0 means identical meaning, 0.0 means completely

unrelated. pgvector's <=> operator computes cosine distance; we convert with `1 - distance` to get similarity.

Q59. What is the difference between simple RAG and CRAG? Simple RAG always trusts retrieved chunks and generates an answer. CRAG grades each chunk for relevance, discards poor ones, rewrites the query if needed, and re-retrieves — giving more accurate, relevant answers.

Q60. What is grounding in AI and how does your system achieve it? Grounding means keeping AI answers tied to real source material. Our system achieves it by only sending retrieved document chunks as context to the LLM with strict instructions never to invent information outside the provided context.

Q61. What is hallucination in LLMs? Hallucination is when an LLM generates confident but false information. Our RAG system prevents this by providing relevant document chunks as context and instructing the LLM to say "I could not find this" if the answer isn't in the context.

Q62. What is prompt engineering in your project? We carefully craft system prompts for each LLM task. For extraction we say "NEVER invent values." For grading we specify exact JSON format. For answers we enforce "only from context" and "include page citations." These constraints control LLM behavior precisely.

Q63. What is the `asynccontextmanager` lifespan in FastAPI? It replaces old startup/shutdown events. The code before `yield` runs on startup (setup logging, create directories, enable LangSmith), and code after `yield` runs on shutdown (close DB connections).

Q64. What is soft delete and why use it? Soft delete sets `deleted=True` instead of removing the row from the database. This preserves data for audit trails, allows recovery, and maintains referential integrity. All queries filter with `Document.deleted == False`.

 SYSTEM DESIGN QUESTIONS

Q65. How would you scale this system to handle 10,000 PDFs daily? Replace `BackgroundTasks` with Celery + Redis for distributed task queues. Add Redis

caching for embeddings. Use connection pooling for DB. Deploy multiple FastAPI workers behind a load balancer. Use S3 for file storage instead of local disk.

Q66. How would you improve RAG accuracy? Implement hybrid search (BM25 keyword + vector semantic). Add cross-encoder reranking of retrieved chunks. Use larger embedding models. Fine-tune chunk size per document type. Add conversation memory for multi-turn Q&A.

Q67. How would you add streaming to the chat endpoint? Replace `response = await rag_chain.ainvoke()` with `async for chunk in rag_chain.astream()`. In FastAPI, return `StreamingResponse` with `text/event-stream` content type to send tokens to the frontend as they arrive from Groq.

Q68. Why pgvector instead of Pinecone or Chroma? pgvector keeps vectors in the same database as metadata, enabling SQL joins like "find documents uploaded this week with similarity > 0.8". No extra infrastructure, ACID transactions, and scales well up to tens of millions of vectors with HNSW.

Total: 68 Questions covering all aspects of the project Each answer is 2-3 lines — easy to describe in interview